System Design: Healthcare Data Normalization Engine

This document explains what the system does, how it is built, and why each decision was made. Numbers, names and limits are taken from the implementation at the time of writing. The tradeoff analysis is in the separate Tradeoff Analysis document.

Contents

1. The problem and why I chose it
2. How I approached it
3. Scope
4. Architecture
5. Processing: the pipeline, sync and async
6. Data model
7. API design
8. Normalization rules
9. Duplicates, matching and conflicts
10. Failure handling
11. Data storage
12. Scalability
13. Multi-tenancy
14. Security
15. Observability
16. Evaluation
17. Deployment
18. Assumptions, limitations and future work

1. The problem and why I chose it

Hospitals, clinics and labs describe the same patient in different ways. One sends `03/04/1990` and pounds; another sends `1990-04-03` and kilograms; a third sends `KUMAR, Ravi` and sex code `1`. Anything built on top of that data, from analytics to AI models to a clinician's screen, needs one consistent format first. This service accepts records from different sources, converts them into one standard format based on HL7 FHIR, validates them, finds duplicates and conflicting values, and keeps a complete history of where every value came from.

Why I chose Problem 1. I had built SynthFix, a tool that finds and repairs problems in machine-learning datasets, so messy data was familiar ground. Building this showed me the most important difference between the two: in an ML dataset you can drop a bad row or overwrite a value and the model is fine. In healthcare that is dangerous. A silently "fixed" birth date or a merge of two patients can put the wrong history in front of a doctor. That difference shaped every decision below.

Design principles, in order of priority:

1. **Never lose accepted input.** Every record that passes basic request checks is stored exactly as received, before normalization starts. Requests that are malformed (invalid JSON, unknown source, oversized) are rejected with an error and not ingested.
2. **Never silently change clinical data.** Every change is logged with the rule and rule version that made it. Invalid records are quarantined, never dropped.
3. **When unsure, stop and ask a human.** Ambiguous dates are rejected, not guessed. Uncertain patient matches go to review, not to an automatic merge.
4. **Isolation is enforced by the database**, not only by application code.
5. **Keep it simple until scale requires otherwise.** One database, one codebase, no extra infrastructure in the MVP.

2. How I approached it

1. **Studied how the industry solves this.** Integration engines such as Mirth and Rhapsody route and transform hospital messages, with a channel per source and error queues for reprocessing. Microsoft's FHIR Converter uses one template per source format. Master Patient Index (MPI) systems match patients with the probabilistic Fellegi-Sunter method and send uncertain pairs to people. FHIR, LOINC and UCUM give a standard data shape, test codes and units. My design is a deliberately small version of that same architecture.
2. **Audited my own SynthFix code** to decide what to reuse: immutable raw data, content hashing and isolated per-detector failures. I also listed what to avoid: guessing data types (which destroys leading zeros in IDs), dropping bad rows, and resolving conflicts by majority vote.
3. **Wrote the design first, then checked it against production practice.** That review caught real gaps before any code existed. Postgres row-level security is skipped by a table's owner unless forced. A plain `SET` leaks one org's id into the next request on a pooled connection. And an updated record must be treated as a new version, not a duplicate.
4. **Questioned the matching rules.** The first draft would have automatically merged two different people with the same name, birth date and gender, which is common for popular names. That led to the strong-identifier rule and the verification step in section 9.
5. **Chose plain servers over serverless.** I considered Vercel, but it cannot run an always-on worker and caps request bodies at 4.5 MB, so it would have forced a redesign. A standard Docker setup runs the same way on a laptop and on any server, such as AWS. I kept the project local for review and included a production configuration instead of deploying it.
6. **Built it, checked it against every requirement in the assignment, and tested the whole setup from a fresh clone** on both Windows and Linux.

Implementation took about 5 hours over 14–15 October 2026, after the design was complete.

3. Scope

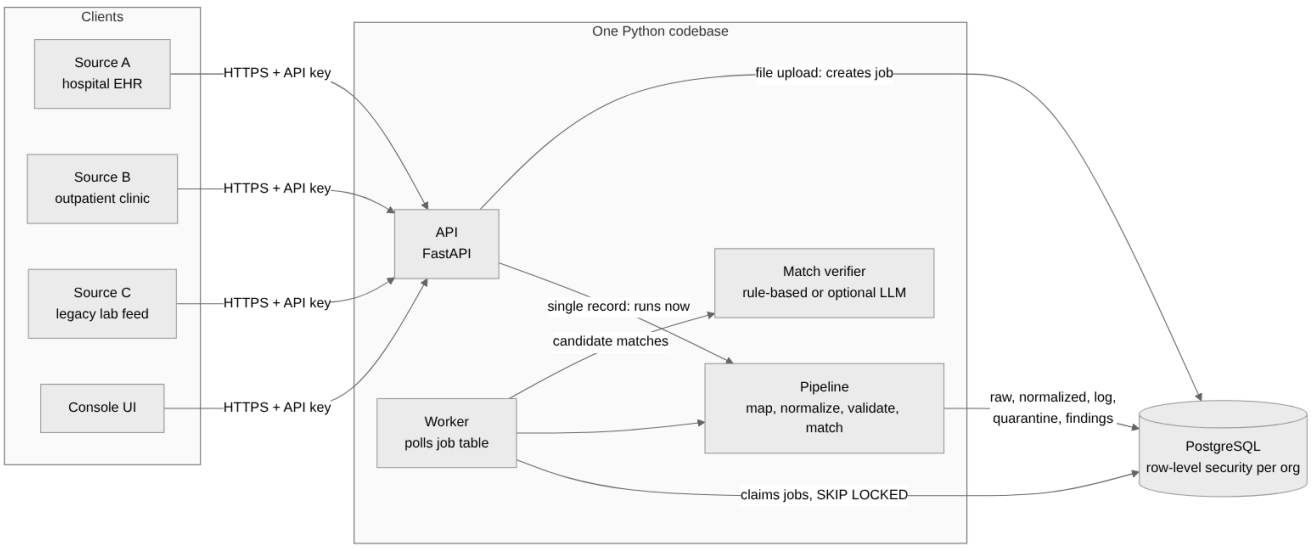
	In scope	Out of scope (see section 18)
Record types	`Patient` and `Observation`	Encounters, medications, conditions

	In scope	Out of scope (see section 18)
Observations	Body weight, body height, blood glucose, heart rate	Blood pressure (two values in one observation)
Input formats	Single JSON records, CSV or JSON-array file uploads	HL7v2, C-CDA, FHIR bundles
Output format	FHIR-lite: FHIR R4 field names and codes, only the fields needed	Full FHIR conformance
Data	Synthetic only	Real patient data

The three synthetic sources are deliberately inconsistent, so every normalization rule is exercised:

	Source A: hospital EHR	Source B: outpatient clinic	Source C: legacy lab feed
Time zone	America/New_York	America/Chicago	America/Los_Angeles
Date format	`MM/DD/YYYY`	`DD-MM-YYYY`	`YYYY-MM-DD`, `DD/MM/YYYY` or `MM/DD/YYYY`
Name	Full name in one column, given name first	Separate first and last columns	`FAMILY, Given`
Sex codes	M, F, U	Male, Female, Other, Unknown	0, 1, 2 (ISO/IEC 5218)
Identifiers	MRN, SSN, phone, email, ZIP	Patient id, insurance member id, mobile, email, pincode	Hospital number, SSN, phone, postcode
Test codes	WT, HT, GLU, HR	BODYWT, HEIGHT, FBS, PULSE	LOINC codes
Units in data	Pounds, mg/dL	kg, mmol/L	kg, cm

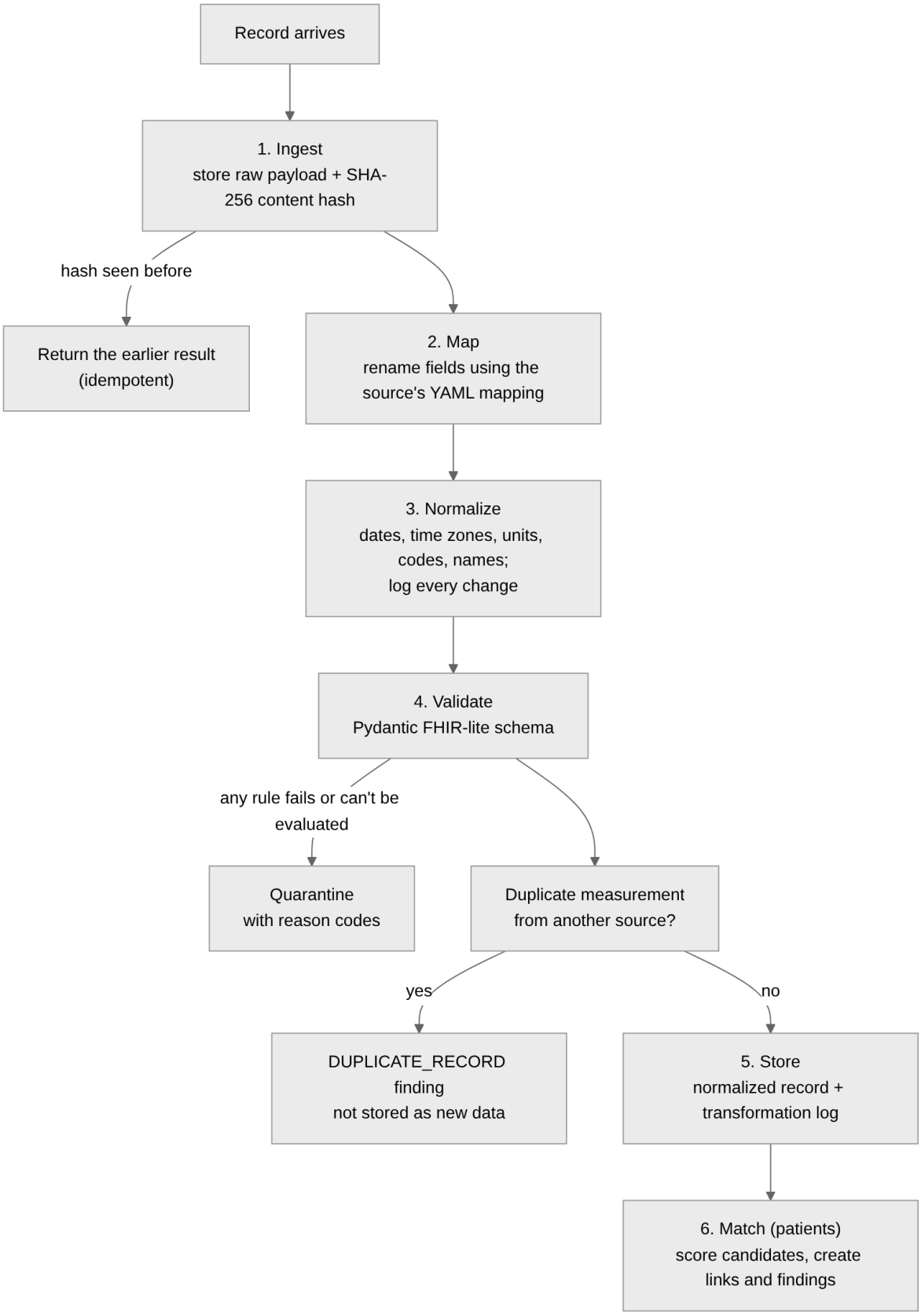
4. Architecture



Single records run through the pipeline inside the API request. File uploads become rows in a job table, and a worker processes them in the background. Both paths use the same pipeline code, so behaviour never differs between them.

Component	Responsibility
API (FastAPI)	Authenticates API keys, enforces body and rate limits, runs the pipeline for single records, creates upload jobs, serves read endpoints, RFC 9457 errors
Worker	Claims jobs with `SKIP LOCKED`, processes uploads in 500-row chunks, runs reprocessing and match verification, retries transient errors, takes over jobs from crashed workers
Pipeline (`app/pipeline/`)	Pure functions with no web or database code: mapping, normalization, validation, match scoring. Easy to unit-test
Services (`app/services/`)	Storage and orchestration: ingest, versions, quarantine, matching, audit log, API keys
Match verifier	Final check on candidate matches; can only veto. Rule-based by default, optional LLM (section 9)
Mapping files (`mappings/`)	One versioned YAML file per source: field names, date formats, time zone, codes
PostgreSQL	All state: raw and normalized records, history, quarantine, matches, jobs, audit log. Isolates orgs with forced row-level security
Console UI	Browser interface for trying records, uploads, quarantine, match review and record lookup

5. Processing: the pipeline, sync and async



The raw payload is stored first, so even a crash halfway through loses nothing: the record can be reprocessed from its raw copy.

Synchronous vs asynchronous

	Single record: `POST /v1/records`	**File upload: `POST /v1/uploads`**
Mode	Synchronous: the pipeline runs inside the request	Asynchronous: returns `202` and a job id immediately; the worker processes it
Response	`201` stored, `200` already seen or duplicate, `422` quarantined with reasons	Poll `GET /v1/jobs/{id}` for status and row counts
Latency	Milliseconds; the caller gets the answer at once	Higher overall, but the request itself returns instantly
Reliability	Raw record saved first; retrying is safe because of the content hash	Job state lives in Postgres, so it survives restarts; each row is isolated, so one bad row never fails the file
Why	A single record is fast and the sender usually wants the answer now	A 100,000-row file would exceed HTTP timeouts and tie up the server

Match verification is always asynchronous: a candidate match waits as `pending_verification`, and nothing treats the two records as one person until the verifier answers.

6. Data model

Canonical formats (FHIR-lite)

These examples use a simplified schema based on FHIR R4: FHIR field names, code systems (LOINC, UCUM) and gender values, but not full FHIR structures. For example, `name` is a single object rather than a list, and identifier `type` is a plain code rather than a `CodeableConcept`. Full FHIR conformance is out of scope.

```
{
  "resourceType": "Patient",
  "identifier": [
    { "type": "MR", "system": "urn:medvora:source-a:mrn", "value": "00123" },
    { "type": "NI", "system": "http://hl7.org/fhir/sid/us-ssn", "value": "123456789" }
  ],
  "name": { "family": "Kumar", "given": ["Ravi", "S"] },
  "gender": "male",
  "birthDate": "1990-04-03",
  "telecom": [{ "system": "phone", "value": "..." }, { "system": "email", "value": "..." }],
  "address": { "postalCode": "..." }
}
```

```
{
  "resourceType": "Observation",
  "code": { "system": "http://loinc.org", "code": "29463-7", "display": "Body weight" },
  "valueQuantity": { "value": 69.85, "unit": "kg", "system": "http://unitsofmeasure.org",
"code": "kg" },
  "effectiveDateTime": "2026-09-12T14:30:00Z"
}
```

Identifier types follow FHIR: `MR` is the source's own patient number, `NI` a national id, `MB` an insurance member number. Identifiers are always strings, so leading zeros survive. Phone and email are stored as `telecom` entries. Numbers are exact decimals end to end, including in API responses.

Tables

Every table has row-level security (section 13). Every table except `orgs` has an `org_id`; `orgs` is filtered by its own `id`.

Table	Purpose	Key columns
`orgs`	One row per customer organization	`id`, `name`
`api_keys`	Credentials	`key_prefix` (unique), `key_hash` (HMAC-SHA256), `role`, `revoked_at`
`raw_records`	Every payload exactly as received; never updated or deleted	`source`, `record_type`, `source_record_id`, `content_hash`, `payload`, `received_at`, `job_id`, `row_number`; unique on (`org_id`, `content_hash`)
`records`	Normalized output, one row per version	`raw_record_id`, `source_record_id`, `mapping_version`, `resource`, plus indexed match keys (`birth_year`, `family_prefix`, `family_key`, `given_key`) and observation keys (`subject_key`, `code`, `effective_at`)
`transformation_log`	One row per field change	`record_id`, `position`, `field`, `source_field`, `rule`, `rule_version`, `original`, `result`
`quarantine`	Records that failed	`raw_record_id`, `reason_codes`, `errors`, `error_class`, `last_error`, `attempts`, `status`, `first_seen_at`, `resolved_record_id`
`match_links`	Candidate and decided patient pairs	`record_id`, `candidate_id`, `points`, `breakdown`, `facts`, `status`, `rule_version`, `verifier`, `verdict`, `decided_by`
`findings`	Conflicts and warnings	`kind`, `code`, `record_id`, `other_record_id`, `field`, `details`
`ingest_jobs`	Upload, reprocess and verify jobs	`kind`, `content`, `status`, row counts, `attempts`, `heartbeat_at`, `last_error`
`audit_log`	Who read or changed what	`api_key_id`, `action`, `resource_type`, `resource_id`, `request_id`, `occurred_at`

Versions. When a source resends a record with the same `source_record_id` but different content, it becomes a new version; the old one is kept. The current version is computed at query time (newest `received_at`), not stored as a flag, because records are append-only. `received_at` uses `clock_timestamp()` rather than `now()`, because `now()` is fixed for a whole transaction, and two versions in the same upload chunk would otherwise tie.

7. API design

REST, JSON, versioned under `/v1`. Every request needs an `X-API-Key` header; the key determines the org, so callers never pass an org id. Interactive documentation is generated at `/docs`.

Method	Path	Role	Success
`GET`	`/health`, `/health/ready`	none	`200`; ready returns `503` if the database is down
`GET`	`/v1/me`, `/v1/sources`	any	`200`

Method	Path	Role	Success
`POST`	`/v1/records`	ingest, admin	`201` new, `200` already received or duplicate, `422` quarantined
`POST`	`/v1/validate`	ingest, admin	`200` with `valid: true/false`; stores nothing
`POST`	`/v1/uploads`	ingest, admin	`202` with a `Location` header
`GET`	`/v1/jobs/{id}`	any	`200`
`GET`	`/v1/records/{id}`, `/lineage`, `/versions`	reader, admin	`200`
`GET`	`/v1/patients/{id}/matches`	reader, admin	`200`
`GET`	`/v1/matches?status=review`	reader, admin	`200`
`POST`	`/v1/matches/{id}/decision`	admin	`200`
`GET`	`/v1/quarantine`, `/v1/quarantine/{id}`	reader, admin	`200`
`POST`	`/v1/quarantine/reprocess`	admin	`202`
`GET`	`/v1/findings`	reader, admin	`200`
`GET`	`/metrics`	admin	`200`

Errors follow RFC 9457 ("Problem Details"), so every error has the same shape, with `type`, `title`, `status`, `detail`, `instance`, the request id and, for quarantined records, the reason codes. Error bodies never echo submitted values. Status codes used: 400, 401, 403, 404, 405, 413, 415, 422, 429, 500, 503. Malformed requests get `400`; `422` means only "the record was valid JSON but failed the healthcare rules".

Pagination uses an opaque cursor (default 50 items, maximum 200), because page numbers shift when new rows arrive while someone is browsing.

8. Normalization rules

Each rule is a small pure function. It returns the new value plus a log entry, or a reason code that sends the record to quarantine. Every rule has a version, stored with each change.

Dates and times

- Dates parse only with the formats the source declares. If a source declares several formats and a value fits two of them differently (`03/04/1990`), it is quarantined as `DATE_AMBIGUOUS`; the system never guesses.
- Timestamps are read in the source's declared time zone and stored in UTC. An explicit offset in the value wins. Local times that don't exist (skipped by a daylight-saving change) or occur twice are quarantined.

Units (all arithmetic in `Decimal`; only converted values are rounded, once, half-up)

Observation	LOINC	Canonical unit	Conversions (rule, version 1)	Rounded to
Body weight	29463-7	kg	lb × 0.45359237; g × 0.001	0.01

Observation	LOINC	Canonical unit	Conversions (rule, version 1)	Rounded to
Body height	8302-2	cm	in × 2.54; m × 100	0.1
Glucose	14749-6	mmol/L	mg/dL ÷ 18.016	0.1
Heart rate	8867-4	/min	none (bpm accepted)	1

Glucose arrives under two LOINC codes: 2345-7 (mass, mg/dL) and 14749-6 (moles, mmol/L). Converting the value also changes the code to 14749-6, so the result never claims mg/dL under a mmol/L code.

Codes and names. Sex codes map from each source's table to FHIR values. Names are split by the source's declared order; if the split is uncertain, the full text is kept and a warning is recorded. Identifiers have separators removed and are upper-cased, and stay strings.

Quarantine reason codes: `DATE_UNPARSEABLE`, `DATE_AMBIGUOUS`, `DATE_OUT_OF_RANGE`, `TIMESTAMP_UNPARSEABLE`, `TIMESTAMP_AMBIGUOUS`, `TIMESTAMP_NONEXISTENT`, `TIMEZONE_MISSING`, `UNIT_UNKNOWN`, `UNIT_INCOMPATIBLE`, `VALUE_NOT_NUMERIC`, `CODE_UNMAPPED`, `IDENTIFIER_MISSING`, `FIELD_MISSING`, `FIELD_INVALID`, `FIELD_UNEXPECTED`, `INTERNAL_ERROR`.

9. Duplicates, matching and conflicts

"Duplicate" can mean four different things, and each is handled differently:

Kind	Example	How it is handled
Same payload sent twice	A client retries after a timeout	Same content hash: the earlier result is returned and nothing is stored again. The hash is SHA-256 over the source name, record type and full payload, and the payload includes the source's own record id (MRN, observation id). So two different records only share a hash if they are byte-for-byte the same record
Same record, new content	A hospital sends a corrected record	Different content gives a different hash, so it is stored. The same `source_record_id` links it as a new version; older quarantined versions are marked superseded
Same measurement from two sources	A lab and a clinic both send one glucose test	Same patient, test code and time: a `DUPLICATE_RECORD` finding; not stored as a new measurement
Same person in two systems	Ravi visits a hospital in January and a clinic in June	Patient matching (below)

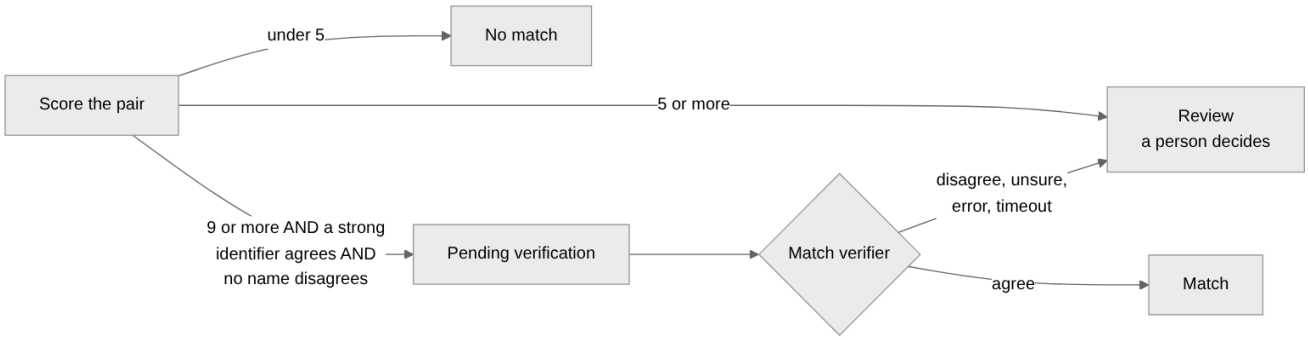
Patient matching

Each candidate pair is described as **comparison facts**, such as `birth_date: agree` or `phone: missing`, and scored. The approach is a simplified Fellegi-Sunter model: agreeing fields add evidence, disagreeing fields subtract it, and agreement on a rare value counts more than on a common one.

Field	Agree	Partial	Disagree
National id	+6		−10
Insurance member id	+4		−2
Birth date	+4	day and month swapped: +2	−2

Field	Agree	Partial	Disagree
Family name	+3, less if common	one edit apart (5+ letters): +1	−1
Given name	+2, less if common (nickname counts as agree)		−1
Phone (last 10 digits)	+3		0
Email	+3		0
Gender	+1		−1
Postal code	+1		0

- **Missing values score 0**, because hospitals collect different fields; a missing field is not evidence of anything. IDs from different issuing systems also count as missing.
- **Common names count less.** A name's frequency is its share of the org's patients (with a minimum base of 200, so small orgs don't make every name look rare). Under 1% earns full points, 1–5% one point less, 5% or more two less (minimum 1).
- **Blocking:** only records sharing a birth year or the first three letters of the family name are compared. In the evaluation, the 938 patient records form 439,453 possible pairs; blocking cut that to 24,354.



- **Strong-identifier rule.** An automatic match needs at least one of national id, insurance member id, phone or email to agree. Name, birth date and gender alone can only reach review. The idea comes from the patient-safety practice of checking at least two identifiers before acting on a patient's identity. It means two different people named Ravi Kumar, born on the same day, go to review instead of being merged automatically.
- **Twins rule.** If the given or family name disagrees, the pair can go no further than review. Twins share a surname, birth date, address and often a phone, so a shared phone alone must not merge them.
- **The match verifier** checks every candidate before it is linked. It receives only the comparison facts, never names, dates or numbers, and answers `agree`, `disagree` or `unsure`. Only `agree` confirms the match. A timeout (20 s), an error, an invalid reply, or a reply citing a field that wasn't in the facts all count as `unsure`.
  - **What the verifier can and can't do.** It only sees pairs the scoring rules already proposed, so it can block a candidate but can never promote a pair the rules didn't propose. A wrong `disagree` costs reviewer time. A wrong `agree` lets through a match the rules proposed, which leaves the system exactly as accurate as the rules alone. The main protection against false matches is therefore the thresholds and the strong-identifier and twins rules; the verifier is an extra safety layer on top of them, not a guarantee.
  - Default: a rule-based verifier, which disagrees if the middle initial, postal code or gender conflict.
  - Optional: an LLM verifier using `claude-haiku-5-5` with structured JSON output and no fallback model. It is off by default (section 14 covers its data handling).
- **Possible namesakes.** Same name and birth date but different national ids produces a `POSSIBLE_NAMESAKE` warning, as does a reviewer deciding "different people" for such a pair. Anyone viewing either record is

warned to check identifiers, like the warning EHRs show.

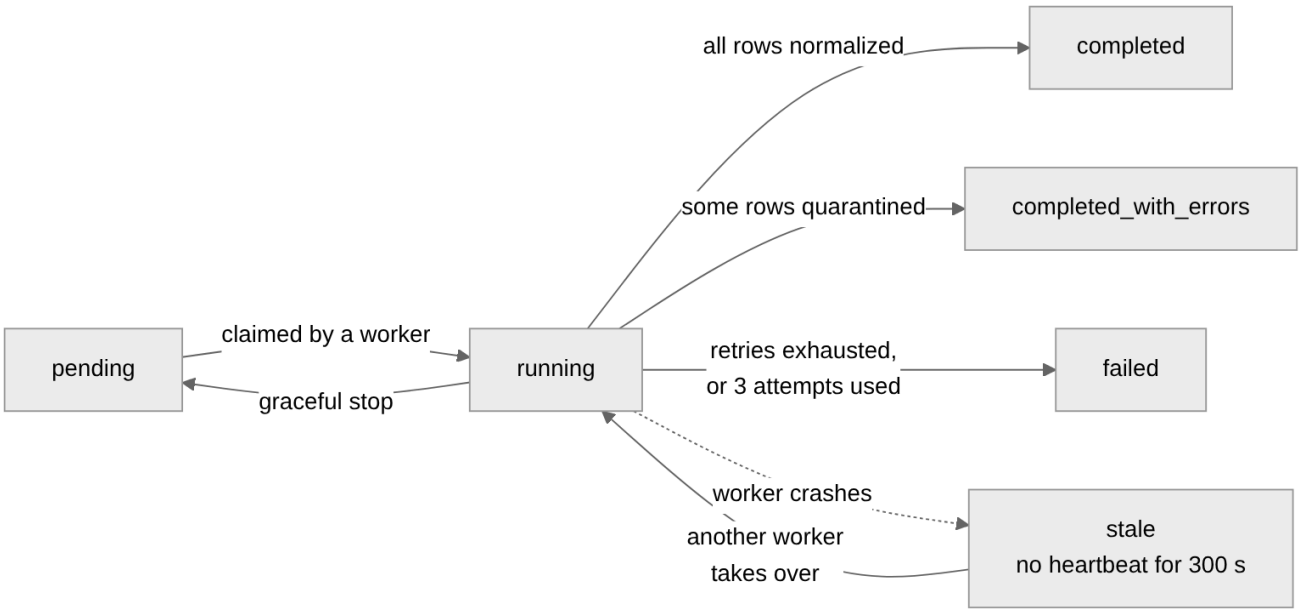
- **Conflicts.** When records in a match disagree on birth date, gender or family name, a conflict finding stores both values and both sources. The system never picks a winner; that is a human decision.

10. Failure handling

Failure	What happens
Malformed request	`400` with a precise reason; nothing stored
Record fails a rule, or a rule can't be evaluated	Raw kept; quarantined with reason codes (`error_class: permanent`); never retried automatically, because it would fail the same way. The system fails closed: when in doubt, it blocks
Unexpected exception on one row	That row is quarantined with `INTERNAL_ERROR` (`error_class: internal`); the rest of the chunk continues
One bad row in a file	Each row runs in its own savepoint, so only that row rolls back; the job ends `completed_with_errors`
Database connection lost or timed out	The whole 500-row chunk is retried after 1 s, 4 s, then 16 s, outside any open transaction. If all retries fail, the job is marked `failed` and can be resumed. Good data is never quarantined because of an outage
Worker crashes mid-job	Progress is saved after every chunk. A job whose heartbeat is older than 300 s is taken over by another worker and resumes where it stopped. After 3 attempts it is marked `failed`
Worker stopped normally	It finishes the current chunk, returns the job to `pending`, and doesn't count the attempt
Two workers, one job	`FOR UPDATE SKIP LOCKED`: each job is claimed by exactly one worker
Same request sent twice	Content hash: the original result is returned
Verifier unavailable	Counts as `unsure`; the pair goes to review

Reprocessing. After a mapping is fixed, `POST /v1/quarantine/reprocess` re-runs quarantined records from their raw copy with the current rules, as a background job. Successes link back through `resolved_record_id`; nothing is deleted.

Job lifecycle



11. Data storage

One PostgreSQL database for everything. Transactions let a normalized record, its full change log and its findings be written together or not at all, so no record ever appears without its history. JSONB holds payloads that differ by source. Row-level security provides tenant isolation. And a table plus `SKIP LOCKED` works as a reliable job queue, so the MVP has no separate broker to run.

Raw vs processed data

- **Raw is the source of truth.** `raw_records` is append-only, and the database enforces it: the application's role has no `UPDATE` or `DELETE` on it.
- **Normalized data is derived.** It can be regenerated from raw at any time with the current rules. Normalized records are also append-only; a change is a new version.
- **History is permanent.** The transformation log, findings and audit log are insert-only for the application.

Retention. The MVP keeps everything. In production, retention would follow each customer's legal requirements, and old raw payloads could move to cheaper object storage while their hashes and lineage stay in Postgres.

12. Scalability

The API and worker are stateless; all state is in Postgres. So each part scales independently when it becomes the bottleneck.

Stage	Signal to change	Change
MVP	—	One API, one worker, one Postgres
More traffic	API latency rises, job backlog grows	More API and worker processes; `SKIP LOCKED` already supports many workers
Heavy batch volume	Job polling loads Postgres	Move jobs to a managed queue (SQS); the job runner is already a separate module

Stage	Signal to change	Change
Large data	Hundreds of millions of rows	Partition raw records and the change log by org and month; move old raw payloads to object storage; read replicas for lookups
Slow matching	Candidate pairs grow with patients	Stronger blocking keys, precomputed match keys, eventually a dedicated MPI service
Real-time feeds	Hospitals stream events	A streaming ingest path (Kafka or Kinesis) into the same pipeline functions

Many organizations. Work is already separated by org, so it can be split by org. Rate limits are per API key (100 records per minute, 10 uploads per hour), so one heavy client can't starve the others. Uploads are capped at 20 MiB and 100,000 rows.

13. Multi-tenancy

All orgs share one database; every row carries an `org_id`. Isolation is enforced in several independent layers, so it doesn't depend on any one of them being correct:

1. **The org comes from the API key**, never from the request.
2. **Postgres row-level security**, enabled and *forced* on every table, including `orgs`. On tenant tables the policy allows only rows whose `org_id` matches the org set for the current transaction; on `orgs` it allows only the row whose `id` matches. The org is set with `set_config('app.org_id', …, true)`; `true` makes it last only for that transaction, so it can't leak into the next request that reuses the pooled connection. If no org is set, every policy matches nothing.
3. **Separate database roles:**
   - `medvora_app` (the API and worker): not a superuser, no bypass, owns nothing, and has no `DELETE` grant anywhere.
   - `medvora_owner` (migrations only): still filtered, because security is forced.
   - `medvora_system` (cannot log in): owns the only two functions that must look across orgs. One finds an API key by its prefix, before the org is known; the other lets a worker claim the next job. `SECURITY DEFINER` alone would not get past forced row-level security, so this role has two narrow policies of its own: read-only access to `api_keys`, and select/update on `ingest_jobs`. It has no access to any other table. Execute rights on both functions are revoked from everyone except `medvora_app`, and both have a pinned `search_path`.
4. **Other orgs' ids return `404`, not `403`**, so ids can't be probed.
5. **Matching never crosses orgs.** Linking patients between organizations would need consent and a data-sharing agreement.

Access within an org: `ingest` keys send data and can check their own jobs; `reader` keys view records, lineage, matches, quarantine and findings; `admin` keys also decide matches, reprocess quarantine and read metrics.

14. Security

Authentication. API keys are random secrets with a public prefix. Only an HMAC-SHA256 of each key is stored, keyed with a server-side secret (the "pepper"), so a stolen database alone doesn't reveal working keys. Lookup is by prefix, then the hash is compared in constant time. Keys can be revoked.

Authorization. Every request is checked twice: the key's role must allow the action, and row-level security limits data to the key's org.

Data protection

- **In transit:** HTTPS via the Caddy proxy in the production configuration, with HSTS.
- **At rest:** in production, encrypted disk volumes or a managed database with encryption (Amazon RDS).
- **Patient data stays out of logs and errors:** logs record ids, rule names and codes, never names, birth dates or payloads. Validation and database errors are stripped of input values.
- **Audit trail:** reads of patient data and every decision are written to `audit_log` with the key and request id.
- **Input limits:** 1 MiB for normal requests and 20 MiB for uploads, enforced while the body streams, so a request without a `Content-Length` can't bypass them. Also limits on rows, JSON depth, number length, filter length, NUL characters and invalid Unicode.
- **Console:** a strict Content-Security-Policy (no inline scripts), `nosniff`, `no-referrer`, `no-store`. All values are rendered as text, never HTML, and the key lives in `sessionStorage`.

LLM verifier data handling. When enabled, it receives only the comparison facts (for example `birth_date: agree`); no names, dates or numbers ever leave the system. Only the configured model receives data; there is no fallback to another model. The model has no tools, so text hidden in a record can't trigger an action (prompt injection), and its output must match a JSON schema. Only the worker container holds the API key.

Security testing. Automated tests cover authentication (missing, forged and revoked keys), org isolation at both API and database level, input limits, cursor tampering, rate limits, and patient-data leakage into logs and errors. `scripts/security.py` runs bandit (code scan) and pip-audit (dependency vulnerabilities) in CI, plus trivy (container scan) when installed.

Compliance context. A production system would need to meet regulations such as HIPAA in the US or India's DPDP Act, depending on where customers are.

15. Observability

- **Structured JSON logs** with a request id on every line. The id is returned in the `X-Request-ID` header and in error bodies, so a failing request can be traced through all its log lines.
- **`/metrics`** (admin only): processing counts by source and status.
- **Health checks:** `/health` (process up) and `/health/ready` (database reachable).
- **Audit log** for data access and decisions.
- **Not built:** error tracking with alerting (such as Sentry). It would be the first addition for production.

16. Evaluation

`scripts/evaluate.py` generates synthetic patients with known truth, runs them through the real pipeline, and measures the results. It creates 500 synthetic people, who appear as 938 patient records across the three sources (so 510 pairs of records truly belong to the same person), plus their observations: 2,839 rows in total. It adds 5 bad rows per file and 10 deliberate namesake pairs, and uses a fixed random seed, so every run is reproducible.

Measure	Result
Data loss	None: every one of the 2,839 input rows ended as either normalized (2,809) or quarantined (30)

Measure	Result
Normalization accuracy	Birth date, sex, weight, glucose, height: 100%. Family name: 99.4% (932/938)
Matching precision	100%: 453 automatic matches, 0 wrong
Matching recall	88.8% automatic (453 of 510 true pairs); 100% counting pairs sent to review
Namesakes merged	0 of 10
Sent to review	57 true pairs and 9 different-person pairs

Caveat: the same code generated the test data and was tested on it, so these numbers are better than real-world performance would be. They show the logic works as designed. Real accuracy needs labelled real data, and the matching weights would then be tuned on it.

17. Deployment

Local: `docker compose up -d --build` starts Postgres, the API and the worker. The same setup is tested in bash and PowerShell.

Deployment-ready, not deployed. The project is meant to be run locally for review, so I didn't deploy it. It does include a production configuration (`docker-compose.prod.yml` and `deploy/Caddyfile`). That configuration would run the same containers on a single server such as an AWS EC2 instance, behind Caddy. Caddy obtains and renews the HTTPS certificate automatically and adds security headers. Only ports 80 and 443 would be open.

What a real production setup would change:

Part	Single-server setup	Production
Compute	One EC2 instance	ECS Fargate, several API and worker tasks
Database	Postgres container	Amazon RDS with backups, encryption and failover
Jobs	Postgres job table	Amazon SQS
Secrets	`.env` file	AWS Secrets Manager
HTTPS	Caddy	Load balancer with an ACM certificate
Logs	Docker logs	CloudWatch, plus error tracking

18. Assumptions, limitations and future work

Assumptions

- Sources send structured CSV or JSON and can declare their date formats, time zone and codes in a mapping file.
- Each source gives its records a stable id.
- Data volume per org is moderate (up to low millions of records), so one Postgres instance is enough at first.
- Someone at each org reviews uncertain matches and quarantined records.

Known limitations

- **Identifiers are stored in plain text.** National ids and member numbers are protected by encryption at rest, row-level security and access control, but not by field-level encryption. Production should add field-level encryption or tokenization.
- **Findings can't be resolved yet.** Conflicts and warnings are recorded but have no "resolved" workflow.
- **Rate limits** are kept in process memory, so with several API instances each instance counts separately. Failed authentication attempts are not rate-limited; keys are long random secrets, but production should still throttle them.
- **Console security headers** are set on the page itself, not on its static files.
- **Matching weights are hand-set**, not learned from labelled data. The middle initial is used only by the verifier, not in the score.
- **Synthetic data** never includes heart rate, so that conversion path is tested only by unit tests.
- **Not built:** streaming ingest, blood pressure, HL7v2 or FHIR input, error tracking.

Future work, in priority order

1. Field-level encryption or tokenization for identifiers.
2. A resolution workflow for findings, and survivorship rules that choose which value to show, with every choice logged.
3. Matching weights learned from labelled data (Fellegi-Sunter with expectation-maximization).
4. Error tracking and alerting.
5. HL7v2 and FHIR input adapters.
6. A streaming ingest path, and SQS for jobs.